CSED233 · 데이터구조 · 포스텍

Ch5. Trees (1) — 기본 개념 & Binary Tree

1. Tree(트리) 기본 정의

이론

- **Tree**: node(원) + edge(선) 집합
- **Root(루트)**: 특별히 지정된 최상위 노드 (관례상 뒤집어 그림)
- 한 edge는 **child ↔ parent** 두 노드를 연결
- 한 노드는 여러 edge에 연결 가능
- **Terminal(단말 노드) (leaf, external)**: 자식 없는 노드
- **Non-terminal(비단말 노드) (internal)**: 자식 있는 노드

Tree(트리)는 노드(vertex)와 간선(edge)으로 구성된 연결된 비순환 무방향 그래프이다. 비어 있지 않은 트리에 루트를 지정한 rooted tree에서는 루트 이외의 각 노드가 정확히 하나의 부모를 가지며, 루트는 부모가 없다. 임의의 두 노드 사이에는 유일한 단순 경로가 존재하고, 노드가 n개인 트리의 간선 수는 $n-1$개이다.

Recursive Definition

- Base: 단일 노드로 구성된 그래프는 트리이다.

Step: 서로 노드를 공유하지 않는 트리 $T_1, T_2, \cdots, T_k$와 새로운 루트 노드 r을 준비하고, r에서 각 T_i의 루트로 간선을 정확히 하나씩 연결하면 새로운 rooted tree가 된다. 이때 각 T_i는 새 트리의 subtree가 된다.

기호 규칙: 이 노트에서 T는 트리, r은 트리의 루트 노드, $n = |V(T)|$은 트리의 전체 노드 수를 나타낸다. 루트 노드 자체와 전체 노드 수를 혼동하지 않도록 루트에는 r, 노드 수에는 n을 일관되게 사용한다.

2. 가족 관계 & Path(경로)

관계 용어

용어	의미
Parent(부모)/Child(자식)	A가 C의 parent
Ancestor(조상)/Descendant(후손)	경로상 위/아래
Sibling(형제)	같은 부모의 자식들

Path(경로)

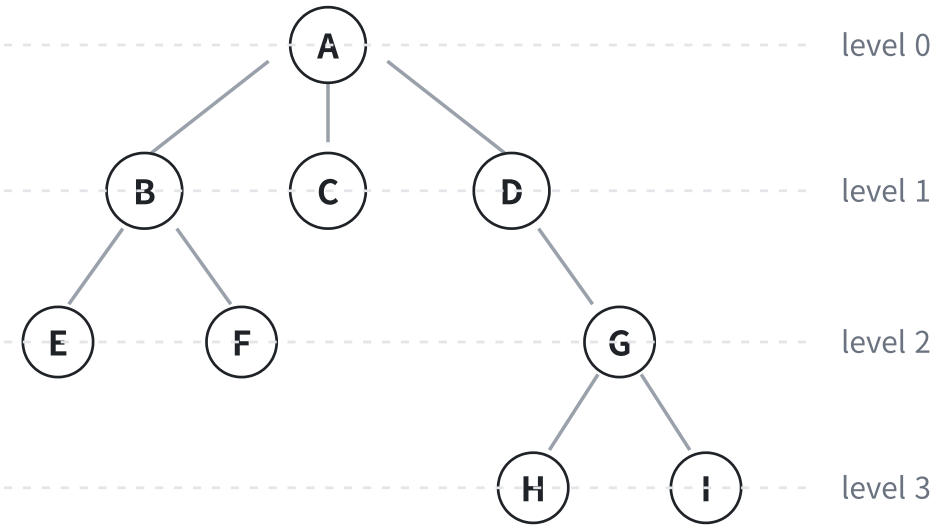
- **Path**: 인접한 두 노드가 간선으로 연결된 노드들의 수열이다. 트리에서는 임의의 두 노드 사이에 유일한 단순 경로가 있으며, 이 경로는 부모→자식뿐 아니라 자식→부모 방향으로도 이동할 수 있다. 부모에서 자식 방향으로만 진행하는 경로는 downward path라고 구분한다. 노드가 k개인 단순 경로의 길이는 간선 수인 k−1이다.
- **Length** = edge 수 = $k - 1$
- 예: §3 도해에서 A→D→G→H는 네 노드와 세 간선으로 구성된 유효한 경로이며, 경로의 길이는 간선 수이므로 3이다.

3. Depth(깊이), Height(높이), Level

정의

개념	정의
Depth(깊이) of M	root → M 까지 path length (= M의 ancestor 수, 자신 제외)
Height(높이) of M	M → 가장 먼 leaf 까지의 path length
Height(높이) of Tree(트리)	height of root
Level	root = level 0, 자식 = level 1, ...

예시



root = level 0, 자식은 부모 level + 1 · Depth of M = 루트→M 경로 길이, Height = 가장 먼 리프까지의 경로 길이

- Depth(G) = 2, Height(G) = 1
- Height(F) = 0 (leaf)
- 그림의 전체 트리를 *T*라고 하면 루트는 A이고, A에서 가장 먼 leaf인 H 또는 I까지의 경로 길이는 3이다. 따라서 Height(*T*)=Height(A)=3이며, Depth(G)=2, Height(G)=1, Height(F)=0이다.

4. Degree(차수) & Ordered Tree(트리)

- **Degree of node** = 자식 수 (leaf → 0)
- **Degree of tree** = max degree among all nodes
- **Ordered tree(순서 트리)**: sibling 간 linear ordering 존재 → 순서가 다르면 다른 tree
- **Forest(포레스트)**: tree들의 집합

5. Binary Tree(이진 트리)

정의

- **Binary tree**: 모든 노드가 **최대 2개** 자식. 각 child는 **left child** 또는 **right child** 로 designate됨.
- → 왼/오 구분이 있으므로 "L만 있는 tree" ≠ "R만 있는 tree"

- Binary tree는 **ordered tree**이다 (L/R 순서 고정).

특수 Binary Tree(이진 트리) 종류

Proper binary tree(정 이진 트리): 모든 노드가 자식을 0개 또는 정확히 2개 가지는 이진 트리이며, leaf의 깊이가 모두 같을 필요는 없다.

Full binary tree(이 노트의 정의: 포화 이진 트리): 높이 h의 모든 레벨 0, 1, ⋯, h가 빈자리 없이 채워진 이진 트리이며, 모든 leaf의 깊이가 h로 같다.

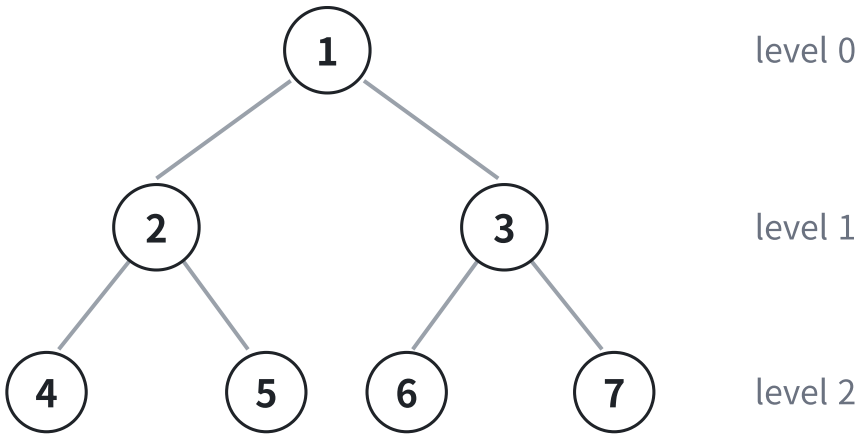
용어 주의: 일부 교재에서는 Proper를 Full binary tree라고 부르고, 이 노트의 Full을 Perfect binary tree라고 부르므로, 문제에서 제시한 정의를 기준으로 구분해야 한다.

- height h인 full binary tree의 노드 수 = $2^{h+1} - 1$
- **유도**: level 0에 1개, level 1에 2개, ..., level h에 2^h개 → 합
  $= 1 + 2 + 4 + \ldots + 2^h = 2^{h+1} - 1$ (기하급수)

6. Node Numbering (Full Binary Tree(포화 이진 트리))

번호 매기기

- 번호 1 (root) ~ $2^{h+1} - 1$
- Level 순 (위→아래), level 내 좌→우



배열 numbering: level 순(위→아래), level 내 좌→우 — 노드 k는 array[k]

번호로 계산되는 성질

연산	공식
Parent(부모) of k	$\lfloor k/2 \rfloor$ (if $k \neq 1$)
Left child of k	$2k$ (if $2k \leq n$)
Right child of k	$2k+1$ (if 2k+1 ≤ n)
Left sibling of k	$k-1$ (if k is odd, k>1)
Right sibling of k	$k+1$ (if k is even, k+1≤n)

- → **이 관계 덕분에 array로 tree 저장 가능** (pointer 없이)
- Right sibling of k: Full 또는 Complete binary tree를 루트부터 1-based level-order로 연속 번호 매겼을 때, k가 짝수이고 k+1≤n이면 오른쪽 형제의 번호는 k+1이다. k+1>n이면 오른쪽 형제는 존재하지 않는다.

7. Complete Binary Tree(완전 이진 트리) (CBT)

정의

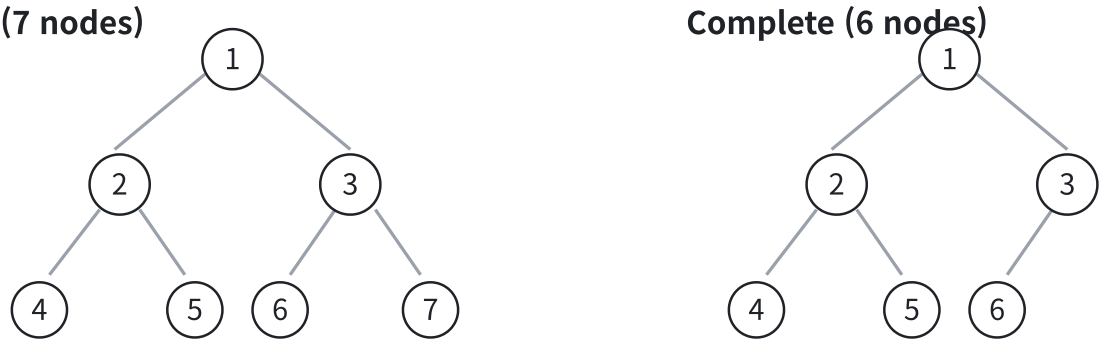
- **Relaxed full binary tree**: height h tree가 complete ⟺
  - Level $0 \sim h-1$ 은 전부 꽉 참
  - Level h (leaf level) 은 **왼쪽부터** 채워짐 (구멍 없음, 오른쪽만 빌 수 있음)
- → full은 complete이지만, 역은 아님

Complete binary tree의 핵심 조건: 높이 h의 Complete binary tree는 마지막 레벨을 제외한 모든 레벨이 완전히 채워져 있고, 마지막 레벨의 노드는 왼쪽부터 빈자리 없이 연속해서 배치된다.

배열 표현이 효율적인 이유: 노드를 루트부터 레벨 순서로 저장하면 Complete binary tree의 모든 노드는 배열 인덱스 1, 2, ⋯, n에 빈칸 없이 저장된다. 따라서 노드 k의 부모는 ⌊k/2⌋, 왼쪽 자식은 2k, 오른쪽 자식은 2k+1로 계산할 수 있으며, 인덱스가 n을 초과하면 해당 자식이 존재하지 않는다.

대표적인 활용: Heap(힙)은 Complete binary tree의 구조적 성질을 이용하므로 별도의 자식 포인터 없이 배열로 효율적으로 구현할 수 있다. 단, Complete binary tree라는 구조적 조건만으로 Heap이 되는 것은 아니며 별도의 heap-order 조건도 만족해야 한다.

시각화



높이 h까지 모든 레벨이 빈자리 없이 채워짐

마지막 level은 왼쪽부터, 구멍 없음

Full은 Complete이지만, Complete가 Full은 아님

Full binary tree(이 노트의 정의: 포화 이진 트리)는 높이 h까지 모든 레벨이 빈자리 없이 채워진 이진 트리이다. 따라서 모든 leaf가 같은 깊이 h에 있으며 전체 노드 수는 $2^{h+1}-1$이다. 반면 Proper binary tree는 각 노드의 자식 수가 0 또는 정확히 2이면 충분하므로, 모든 Proper binary tree가 Full binary tree인 것은 아니다.

→ Complete는 마지막 level이 "왼쪽부터 연속 채움". 중요 사용처: Heap(힙) (다음 Ch7) 구현은 CBT → array 표현 가능.

---

8. Binary Tree(이진 트리) 구현

Array(배열) 구현

- Full binary tree의 numbering을 그대로 사용
- char array[123]; — 노드 k는 array[k]에 저장
- Array 구현에서는 루트를 인덱스 1에 배치하고 왼쪽 자식은 $2k$, 오른쪽 자식은 $2k+1$에 저장한다. Complete binary tree는 노드가 1부터 n까지 연속으로 배치되므로 부모·자식의 위치를 인덱스 계산만으로 구할 수 있다. 일반적인 sparse binary tree에서는 비어 있는 자식 위치도 예약하거나 별도로 존재 여부를 기록해야 하므로 공간 낭비가 커질 수 있다. C/C++의 char array[123]에서는 인덱스 0을 사용하지 않을 경우 인덱스 1부터 122까지만 저장할 수 있으며, 실제 노드가 존재하는지도 확인해야 한다.
- **장점**: pointer 불필요, cache friendly

- **단점**: sparse tree면 공간 낭비 심함

Linked 구현

```
struct node {
  char val;
  node* left;
  node* right;
};
```

- 각 node에 2 pointer
- sparse/dynamic tree에 유리, 일반적

9. Tree(트리) Traversal (순회)

개념

- **Traversal(순회)**: 모든 노드를 **정확히 한 번** 체계적 방문
- 방문 시 action 수행: print, update, check, evaluate 등
- → tree를 **linearize** 하는 방법

DFS (Depth-First Search)

- 깊이 먼저 내려감
- 재귀 (또는 call stack) 기반
- 3가지 variation: **preorder / inorder / postorder**
- Time complexity(시간복잡도): $O(n)$ (모든 노드 1번씩 방문)

10. 세 가지 DFS Traversal(순회)

정의

- **Preorder(전위 순회)**: 노드 → 왼쪽 → 오른쪽

$$\text{Preorder}(T) = \langle n, \text{Preorder}(T_1), \ldots, \text{Preorder}(T_k) \rangle$$

- **Inorder(중위 순회)**: 왼쪽 → 노드 → 오른쪽

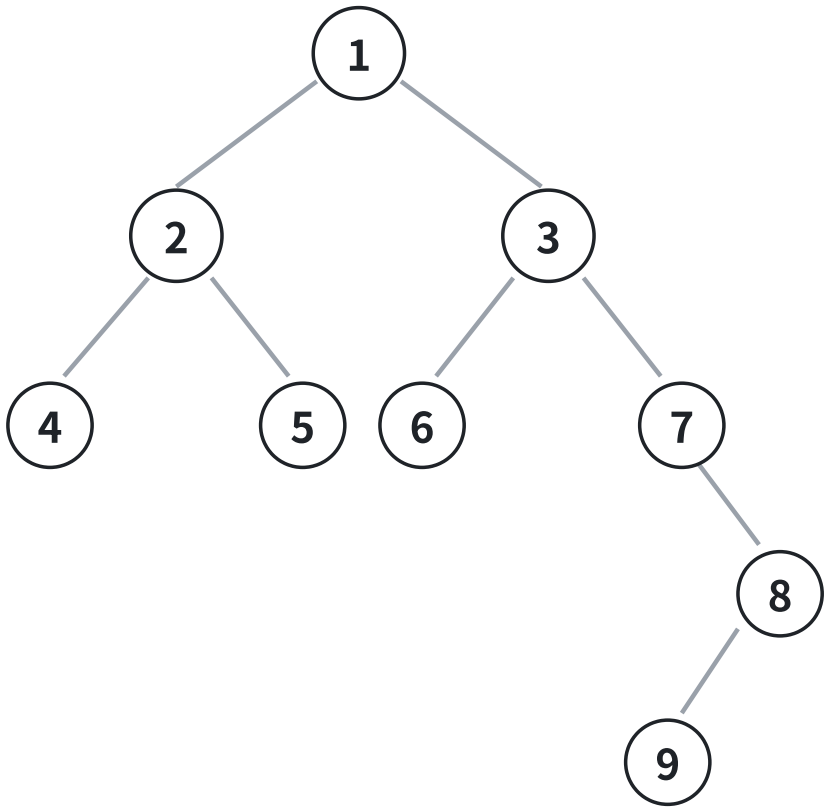
$$\mathrm{Inorder}(T) = \langle \mathrm{Inorder}(T_1), n, \mathrm{Inorder}(T_2), \ldots \rangle$$

- **Postorder(후위 순회)**: 왼쪽 → 오른쪽 → 노드

$$\mathrm{Postorder}(T) = \langle \mathrm{Postorder}(T_1), \ldots, \mathrm{Postorder}(T_k), n \rangle$$

Inorder(중위 순회)는 이진 트리에서 왼쪽 서브트리, 루트, 오른쪽 서브트리 순서로 재귀적으로 방문한다. 이진 트리 T의 루트가 r, 왼쪽 서브트리가 L, 오른쪽 서브트리가 R일 때, Inorder(T)=Inorder(L) $\mathbin{/\!/} \langle r \rangle \mathbin{/\!/}$ Inorder(R)이다. 여기서 $\mathbin{/\!/}$는 수열 연결을 뜻하며, 빈 서브트리의 순회 결과는 빈 수열이다.

예제 Tree(트리)



예제 트리 — 7의 오른쪽 자식 8, 8의 왼쪽 자식 9

- **Preorder**: 1 2 4 5 3 6 7 8 9
- **Postorder**: 4 5 2 6 9 8 7 3 1

- **Inorder**: 4 2 5 1 6 3 7 9 8

Inorder(중위 순회): 왼쪽 서브트리 → 현재 노드 → 오른쪽 서브트리 순서로 방문한다. 예제 트리에서 7의 오른쪽 자식은 8이고, 8의 왼쪽 자식은 9이므로, 전체 Inorder 순회 결과는 4 2 5 1 6 3 7 9 8이다.

Summary

- Tree 기본 용어 (root, parent, child, sibling, ancestor, descendant)
- Path, Depth, Height, Level, Degree
- Binary Tree (proper, full, complete)
- Node numbering & array 저장
- Traversal (preorder / inorder / postorder) — 모두 $O(n)$